

logfile, but passing the type object to `str()` will return a rather messy-looking string. Instead, use the `__qualname__` attribute, which all type objects have, to write a simpler, human-readable string:

```
>>> str(type(42)) # Passing the type object to str() returns a messy string.
"<class 'int'>"
>>> type(42).__qualname__ # The __qualname__ attribute is nicer looking.
'int'
```

The `__qualname__` attribute is most often used for overriding the `__repr__()` method, which is explained in more detail in Chapter 17.

Non-OOP vs. OOP Examples: Tic-Tac-Toe

At first, it can be difficult to see how to use classes in your programs. Let's look at an example of a short tic-tac-toe program that doesn't use classes, and then rewrite it so it does.

Open a new file editor window and enter the following program; then save it as *tictactoe.py*:

```
# tictactoe.py, A non-OOP tic-tac-toe game.

ALL_SPACES = list('123456789') # The keys for a TTT board dictionary.
X, O, BLANK = 'X', 'O', ' ' # Constants for string values.

def main():
    """Runs a game of tic-tac-toe."""
    print('Welcome to tic-tac-toe!')
    gameBoard = getBlankBoard() # Create a TTT board dictionary.
    currentPlayer, nextPlayer = X, O # X goes first, O goes next.

    while True:
        print(getBoardStr(gameBoard)) # Display the board on the screen.

        # Keep asking the player until they enter a number 1-9:
        move = None
        while not isValidSpace(gameBoard, move):
            print(f'What is {currentPlayer}\''s move? (1-9)')
            move = input()
        updateBoard(gameBoard, move, currentPlayer) # Make the move.
```